

Ongoing Platform

Project Research Report - Simple MVP-focused startup, market, and technical analysis

Prepared for: Ongoing Platform MVP planning

Date: June 2026

Current confirmed stack: Next.js, Firebase Authentication, MongoDB, Cloudflare R2, Vercel

Important note: This report challenges the idea honestly. The project is feasible, but the market is competitive. The product should not compete only as another form builder. It should focus on the gap after form submission: simple dashboards, record management, status tracking, and report export.

1. Project Overview

Ongoing Platform is a web-based platform where an admin can create forms, share them with users, collect responses, view automatic dashboards, manage submitted records, and export reports. The user side is simple: a respondent opens a public form link, fills the form, uploads files if needed, submits it, and sees a success message.

The project solves the problem that many form tools are good at collecting responses, but weak when users need to organize, track, visualize, and report the data. Users often export responses into spreadsheets or connect extra tools to get basic dashboards and status tracking.

Target users

- General-purpose users: small businesses, freelancers, educators, recruiters, event organizers, NGOs, agencies, and local teams.
- Admins who need forms plus simple response management, not complex database tools.
- Respondents who only need a clean public form experience without creating an account.

Why people would use it

- It reduces the need to move data from forms into spreadsheets.
- It gives instant dashboards and record tables after submissions arrive.
- It helps non-technical users track records using default statuses such as New, In Progress, and Completed.
- It supports file collection with Cloudflare R2 storage and a simple 100 MB upload limit.

2. Market Validation

This is a real problem. Google Forms provides real-time response charts and export to Google Sheets, but deeper analysis usually still happens in Sheets. That confirms both the demand for response collection and the gap after collection. Typeform also positions itself around collecting more data and then triggering workflows, showing that form data is increasingly tied to business actions, not just surveys.

Market signals are positive. Online form builder software is estimated by WiseGuyReports at about USD 4.22 billion in 2025 and projected to reach USD 10 billion by 2035, with a 9.1% CAGR. The broader no-code development platform market is also growing quickly, with The Business Research Company estimating growth from USD 35.61 billion in 2025 to USD 45.24 billion in 2026.

Evidence people pay

- Typeform, Jotform, Airtable, Fillout, Tally Pro, and Formbricks all have paid plans, proving customers pay for form customization, response limits, reporting, automation, branding, and collaboration.
- Jotform paid plans include higher submission limits, storage, and branding removal.
- Airtable charges per editor seat, showing that teams pay when collected data becomes operational data.

- Formbricks and Form.io show that privacy, self-hosting, APIs, and enterprise workflows can be monetized.

Trends that support the idea

- Non-technical teams want no-code tools.
- Businesses want faster data collection and reporting.
- Small businesses prefer simple tools over complex BI platforms.
- File collection, lead forms, event forms, HR forms, and feedback forms remain common use cases.

Trends that weaken the idea

- The form builder market is crowded.
- Generous free competitors like Google Forms and Tally reduce willingness to pay for basic form creation.
- AI form builders and automation features are becoming standard in larger tools.
- Distribution is harder than development; building the product is easier than getting users.

3. Existing Competitors

Competitor	Features	Pricing model	Strengths	Weaknesses / opportunity
Google Forms	Forms, templates, response summary charts, export to Sheets.	Free with Google account / Workspace ecosystem.	Very easy, trusted, free, familiar.	Limited record management, limited custom dashboards, weak workflow/status tracking. Opportunity: better post-submission management.
Typeform	Beautiful conversational forms, logic, AI, workflows, integrations.	Free + paid response-based plans.	Excellent UX and completion rates.	Can become expensive with response caps. Opportunity: simpler dashboard + records at lower cost.
Jotform	Drag-drop forms, payments, integrations, reports, templates.	Free + paid plans based on forms/submissions/storage.	Very mature and feature-rich.	Can feel heavy for simple users. Opportunity: cleaner MVP focused on records and dashboards.
Airtable	Database, forms, interfaces, automations, views.	Free + per-seat paid plans.	Strong data management and team workflows.	Learning curve and per-seat cost. Opportunity: simpler form-first data management.
Tally	Keyboard-first form builder, unlimited forms/submissions, logic, uploads.	Free + Pro workspace plan.	Very generous free plan and simple UI.	Less focused on record management/dashboarding. Opportunity: stronger response operations.
Fillout	Drag-drop forms, themes, advanced forms, branding, smart routing.	Free + paid response-based plans.	Modern form UX, strong customization.	Competitive on forms; opportunity exists in simpler built-in records/reporting.
Formbricks	Open-source survey/experience management, surveys, logic, API, self-hosting.	Free/open-source + hosted paid plans.	Privacy-first and developer-friendly.	More survey/feedback focused. Opportunity: business form records and reports.
Form.io	Open-source form builder/data management/API platform.	Enterprise/configurati on pricing.	Powerful for developers and enterprise apps.	Too technical for non-technical admins. Opportunity: easier UX.

4. Similar Projects Built Before

Project	Link / status	Lessons learned
Form.io	Open-source core + enterprise platform.	A form builder becomes powerful when connected to APIs, data models, permissions, and storage. But developer-oriented platforms can become too complex for casual users.
Formbricks	Open-source survey platform with hosted plans.	Open-source can help trust and adoption, but monetization still requires clear hosted value, support, and enterprise features.

Project	Link / status	Lessons learned
Tally	Successful modern simple form builder.	Simplicity and generous pricing can win attention. But unlimited free pricing makes it hard for new basic form builders to charge.
Fillout	Growing SaaS form builder.	Better UX, templates, integrations, and smart logic matter. Ongoing Platform must avoid being weaker than these on core form creation.
Airtable	Successful data platform with forms.	Data management is valuable, but users may need education. The UI must hide database complexity.
Google Forms + Sheets workflow	Massive existing habit.	The real competitor is not only a tool, but the habit of exporting to Sheets. Migration and import/export must be easy.

5. Technical Feasibility Analysis

Overall architecture

The project is technically feasible with a modern serverless architecture. The frontend can be built in Next.js 15 using TypeScript, Tailwind CSS, and shadcn/ui. Firebase Authentication handles admin signup/login with Google and email/password. MongoDB stores users, forms, fields, responses, dashboard metadata, and report records. Cloudflare R2 stores uploaded files. Vercel hosts the frontend and API routes/server actions.

Recommended architecture flow

- Admin authenticates through Firebase Auth.
- Next.js server verifies Firebase ID token before accessing protected APIs.
- Admin creates forms; form schema is stored in MongoDB.
- Public user opens a form slug; published form schema is served without login.
- User submits response; metadata and field values go to MongoDB.
- Uploaded files go directly or indirectly to Cloudflare R2 using signed upload URLs.
- Dashboard reads aggregated response data from MongoDB.
- PDF/CSV report generation runs through server-side API logic.

Database design - main MongoDB collections

Collection	Purpose	Important fields
users	Admin profile linked to Firebase UID.	firebaseUid, email, name, avatar, createdAt
workspaces	Single-admin workspace for MVP.	ownerId, name, logoUrl, branding, limits
forms	Main form document.	workspaceId, title, description, status, slug, theme, settings
form_fields	Field definitions or embedded field array.	formId, type, label, required, options, order, conditionalRules
responses	Submitted form data.	formId, submittedAt, status, answers, metadata
response_files	Uploaded file metadata.	responseId, fieldId, r2Key, fileName, fileSize, mimeType
templates	Starter form templates.	name, category, fields, theme, isActive
reports	Generated report metadata.	formId, reportType, generatedAt, fileUrl

APIs required

- Auth middleware: verify Firebase token.
- Forms API: create, edit, duplicate, publish, close, delete.
- Fields API: add, update, reorder, remove fields.
- Public form API: fetch published form by slug.

- Submission API: validate and save response.
- Upload API: create signed upload URL for Cloudflare R2.
- Responses API: list, search, filter, update status.
- Dashboard API: return aggregate charts and metrics.
- Export API: CSV and PDF generation.

Scalability concerns

- MongoDB response documents can grow large if answers are stored without limits.
- Dashboard aggregation can become slow on high-volume forms without indexes or precomputed summaries.
- File uploads can create bandwidth and security issues if not handled with signed URLs and file scanning rules.
- PDF generation can be slow if done synchronously for large response tables.

6. Developer Challenges

Challenge	Why it occurs	Severity	Recommended solution
Dynamic form builder	Fields, validation, reordering, and logic make UI state complex.	High	Keep MVP field types limited to 10 and use a clean schema version system.
Conditional logic	Even basic show/hide rules can break validation and hidden required fields.	Medium	Support only one-level show/hide logic in MVP. Hidden fields should not be required.
File upload security	Users can upload large or unsafe files.	High	Use signed R2 URLs, file type allowlist, 100 MB limit, private bucket, virus scan later.
Firebase + MongoDB auth mapping	Firebase auth users must map correctly to MongoDB users/workspaces.	Medium	Use firebaseUid as stable unique key and verify tokens server-side.
Dashboard performance	Charts require aggregation over response data.	Medium	Index formId/submittedAt/status and precompute counters after each submission.
PDF generation	Large reports can timeout on serverless platforms.	Medium	Limit MVP PDF size; export large data as CSV; use background jobs later.
Public form spam	Public links can receive bot submissions.	Medium	Add rate limiting, honeypot, optional CAPTCHA later.
Multi-tenant isolation	Admin must never access another admin's data.	High	Every query must filter by workspaceId and authenticated ownerId. Add tests.
Cost overrun	Storage and serverless compute costs increase with files and exports.	Medium	Set upload limits, plan limits, usage monitoring, and spend alerts.

7. Hidden Problems Most Founders Miss

- Legal and privacy: form responses may contain personal data, resumes, phone numbers, IDs, or health information. Privacy policy, deletion tools, and data export are needed early.
- Abuse: public forms can be used for spam, phishing, or collecting sensitive data. Terms of use and abuse reporting are needed.
- Data retention: users will expect backups and recovery if form data is deleted accidentally.
- Customer support: non-technical users ask many questions about forms, reports, and lost submissions.
- Migration friction: many users already use Google Forms. Importing forms/responses later can help adoption.
- Monetization risk: basic forms are free elsewhere. The paid value must be record management, dashboarding, branding, storage, and reports.
- Retention risk: some users create one form and do not return. Templates and recurring workflows improve retention.
- Compliance: if targeting schools, healthcare, or HR, compliance expectations increase quickly. Avoid regulated vertical claims in MVP.

8. MVP Plan - 30 Days

The smallest launchable MVP should prove one promise: an admin can create a form, share it, collect responses, see automatic dashboard charts, manage record status, and export CSV/PDF.

Core features only

- Admin signup/login with Firebase Google and email/password.
- Dashboard with total forms, total responses, recent responses, and response trend.
- Form list with create, edit, publish, close, delete.
- Form builder with 10 fields: short text, long text, email, phone, number, date, dropdown, multiple choice, checkbox, file upload.
- 5-10 templates plus blank form creation.
- Branding: logo, cover image, primary color.
- Public form page with validation and file upload up to 100 MB.
- Response table with status: New, In Progress, Completed.
- Simple auto-dashboard per form: total responses, trend, status distribution, question charts.
- CSV export and basic PDF summary export.

Postpone

- Team members and roles.
- Kanban/calendar/gallery views.
- Advanced dashboard builder.
- Automation builder.
- AI insights.
- Integrations.
- Mobile app.
- API access.
- Payment forms.

Estimated 30-day timeline

Week	Focus	Output
Week 1	Setup, auth, database schema, design system, dashboard shell.	Working login, protected admin layout, MongoDB connection, base collections.
Week 2	Form builder and templates.	Create/edit/publish forms with 10 fields and branding.
Week 3	Public form, submissions, R2 upload, response table.	Users can submit forms and admins can view/manage records.
Week 4	Charts, exports, testing, polish, deployment.	Auto dashboard, CSV/PDF export, responsive QA, production deployment.

Team size

- Solo developer: possible but tight for 30 days; reduce polish and PDF complexity.
- Ideal small team: 1 full-stack developer + 1 UI/UX designer + part-time QA/tester.
- If only one person builds it, launch a beta with fewer templates and basic reports.

9. Cost Analysis

The estimates below are rough monthly operating estimates for a lean MVP using Vercel, Firebase Authentication, MongoDB Atlas, and Cloudflare R2. Actual costs depend heavily on file upload volume, response volume, PDF generation, and traffic. Cloudflare R2 pricing is attractive because standard storage is listed at USD 0.015/GB-month with no egress fees; MongoDB Atlas has a free tier and shared tiers from about USD 9/month, with dedicated tiers from about USD 60/month; Vercel Pro starts at USD 20/month.

Scale	Assumptions	Estimated monthly cost	Notes
100 users	Light traffic, <10 GB files, small MongoDB data.	\$0 - \$30	Can use Vercel Hobby for testing, MongoDB free/shared, R2 free tier. For commercial use, move to paid Vercel.
1,000 users	Moderate forms, 50-100 GB files, more submissions.	\$40 - \$120	Vercel Pro + MongoDB shared/flex + low R2 storage/ops cost.
10,000 users	High submissions, 500 GB - 1 TB files, more API load.	\$180 - \$700	Need stronger MongoDB tier, caching, optimized aggregation, more monitoring.
100,000 users	Large traffic, multi-TB storage, high exports.	\$1,500 - \$7,000+	Requires database scaling, background jobs, queue, CDN, observability, abuse protection, support team.

Other cost categories

- Development cost: solo/freelance MVP can be low, but a production-quality SaaS usually needs design, development, QA, copy, legal pages, and support.
- Third-party services: email provider, monitoring, analytics, error tracking, CAPTCHA, and PDF rendering may add monthly cost.
- Support cost: grows with non-technical users and file upload issues.

10. Monetization Opportunities

Model	Example	Why it works
Freemium	Free: 3 forms, 100 responses/month, 100 MB total storage.	Allows users to try without risk.
Pro subscription	■ 399-■ 799/month or \$9-\$19/month: more forms, responses, storage, branding removal.	Good for freelancers and small businesses.
Business plan	■ 1,499-■ 2,999/month or \$29-\$59/month: higher limits, custom domain, better reports.	Useful for teams and growing businesses.
Storage add-on	Extra storage packs for file-heavy forms.	Important because file uploads create real cost.
Template marketplace	Sell industry templates later.	Templates help acquisition and monetization.
Enterprise/custom	Custom limits, SSO, compliance, dedicated support later.	Only after MVP proves demand.

11. SWOT Analysis

Strengths	Weaknesses
Clear pain point after form submission; simple MVP scope; general-purpose use cases; modern low-cost stack; dashboards + records provide more value than basic forms.	Crowded market; basic form creation is already free; no team collaboration in MVP; no integrations initially; MongoDB aggregations need careful design.
Opportunities	Threats
Position as simple form response operating system; templates for HR/events/education; affordable alternative for small businesses; file collection and reports as paid value; later add workflows and AI.	Google Forms and Tally are free; Jotform/Typeform/Fillout are mature; users may not switch from existing tools; abuse/spam risk; storage costs and support burden can grow.

12. Success Probability

Honest success probability for an indie MVP: 35% to 45% if launched as a general form builder. The market is too crowded for a basic clone. Success probability improves to 55% to 65% if the positioning is narrow and strong: "forms with automatic dashboards, record tracking, and PDF/CSV reporting for non-technical users."

Biggest reasons it may fail

- Users already have free tools and may not see enough reason to switch.
- The product may become too broad too early.
- Customer acquisition may be harder than building.
- Competitors already have templates, integrations, AI, payments, and brand trust.
- File upload and reporting features may create support and infrastructure cost.

Biggest reasons it may succeed

- The problem after response collection is real.
- Non-technical users like simple dashboards and reports.
- The MVP scope is clear and buildable.
- A simple product can beat complex tools for beginners.
- Templates can help users reach value quickly.

Recommended improvements

- Pick one strong launch use case first, such as event registrations, job applications, feedback forms, or file collection forms.
- Make the first dashboard appear immediately after the first response.
- Create excellent templates instead of many average features.
- Add import from CSV/Google Forms later to reduce switching friction.
- Track usage analytics from day one: created forms, published forms, submissions, exports, and return visits.

13. Final Verdict

Yes, the project is worth building, but only as a focused MVP. It should not be built as a full Google Forms + Airtable + Zapier competitor from day one. The first version should prove one thing clearly: an admin can collect responses and immediately manage and understand them without opening a spreadsheet.

Should it be built?

Yes. Build a 30-day MVP and test with real users. Do not spend months building advanced features before validation.

Who should build it?

- A full-stack developer comfortable with Next.js, Firebase Auth, MongoDB, Cloudflare R2, and PDF/CSV generation.
- A UI/UX designer who can keep the product simple for non-technical users.
- Early testers from small businesses, colleges, freelancers, event teams, and HR users.

What would make it significantly more successful?

- A sharper niche for launch instead of "for everyone" marketing.
- Fast onboarding with 5-10 high-quality templates.
- A beautiful and useful auto-dashboard that appears with zero setup.

- A clean record table with status tracking that feels easier than spreadsheets.
- Simple pricing that makes sense against free competitors.

Final recommendation: Build the MVP, launch quickly, and validate whether users care more about dashboards and record management than just form creation. If users only use it as another form builder, the project will struggle. If users rely on the dashboard, records, and exports, the project has a strong chance to grow.

References

- Google Forms product page: <https://workspace.google.com/products/forms/>
- Typeform pricing: <https://www.typeform.com/pricing>
- Typeform homepage: <https://www.typeform.com/>
- Jotform pricing: <https://www.jotform.com/pricing/>
- Jotform homepage: <https://www.jotform.com/>
- Airtable pricing: <https://airtable.com/pricing>
- Airtable forms documentation: <https://support.airtable.com/docs/building-and-sharing-forms-in-airtable>
- Tally pricing: <https://tally.so/pricing>
- Fillout homepage: <https://www.fillout.com/>
- Formbricks homepage: <https://formbricks.com/>
- Formbricks GitHub: <https://github.com/formbricks/formbricks>
- Form.io homepage: <https://form.io/>
- Form.io open source: <https://form.io/open-source/>
- Online Form Builder Software market estimate: <https://www.wiseguyreports.com/reports/online-form-builder-software-market>
- No-code development platforms market report:
<https://www.thebusinessresearchcompany.com/report/no-code-development-platforms-global-market-report>
- Cloudflare R2 pricing: <https://developers.cloudflare.com/r2/pricing/>
- MongoDB Atlas pricing: <https://www.mongodb.com/products/platform/atlas-cloud-providers/aws/pricing>
- Vercel pricing: <https://vercel.com/pricing>
- Firebase Authentication documentation: <https://firebase.google.com/docs/auth>